UD 7. Formularios

1 Formularios de plantilla

1.1 ngModel

Los formularios de plantilla son la forma más sencilla de trabajar con formularios en Angular.

Se basan en directivas dentro del HTML y usan ngModel para enlazar datos (two-way binding).

formulario.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-formulario',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './formulario.component.html'
})
export class FormularioComponent {
  usuario = { nombre: '', email: '' };

  enviar(form: any) {
    console.log('Formulario enviado:', form.value);
  }
}
```

formulario.component.html

```
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <label>Nombre:</label>
  <input name="nombre" [(ngModel)]="usuario.nombre" required
#n="ngModel">
  <div *ngIf="n.invalid && n.touched">El nombre es
obligatorio.</div>

  <label>Email:</label>
  <input name="email" [(ngModel)]="usuario.email" email
#e="ngModel">
  <div *ngIf="e.invalid && e.touched">Email no válido.</div>

  <button type="submit">Enviar</button>
</form>
```

1.2 ngForm

Permite trabajar con formularios **template-driven**, gestionando estados, validaciones y envío directamente desde la plantilla HTML.

Html

```
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <label>Nombre:</label>
  <input name="nombre" ngModel required #n="ngModel">
  <div *ngIf="n.invalid && n.touched">El nombre es
obligatorio.</div>

  <label>Email:</label>
  <input name="email" ngModel email #e="ngModel">
  <div *ngIf="e.invalid && e.touched">Email no válido.</div>

  <button type="submit" [disabled]="f.invalid">Enviar</button>
</form>
```

Ts

```
usuario = { nombre: '', email: '' };

enviar(form: any) {
  console.log('Formulario enviado:', form.value);
}
```

1.3 Tipos de campo

- **Campo de texto (text)**

```
<input type="text" name="nombre" ngModel required>
```

- **Campo numérico (number)**

```
<input type="number" name="edad" ngModel min="1" max="120">
```

- **Campo de email (email)**

```
<input type="email" name="correo" ngModel email>
```

- **Campo de contraseña (password)**

```
<input type="password" name="clave" ngModel required
minlength="6">
```

- **Área de texto (textarea)**

```
<textarea name="descripcion" ngModel rows="4"></textarea>
```

- **Selector desplegable (select)**

```
<select name="pais" ngModel required>
  <option value="">Seleccione...</option>
  <option value="es">España</option>
  <option value="fr">Francia</option>
</select>
```

- **Casilla de verificación (checkbox)**

```
<input type="checkbox" name="acepta" ngModel>
```

- **Botones de opción (radio)**

```
<label><input type="radio" name="genero" value="hombre" ngModel> Hombre</label>
<label><input type="radio" name="genero" value="mujer" ngModel> Mujer</label>
```

- **Selector de fecha (date)**

```
<input type="date" name="fecha" ngModel>
```

- **Selector de archivo (file)**

```
<input type="file" name="archivo" (change)="subir($event)">
```

Un ejemplo completo con varios tipos de campo

```
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <input type="text" name="nombre" ngModel required>
  <input type="number" name="edad" ngModel>
  <input type="email" name="correo" ngModel>
  <textarea name="comentarios" ngModel></textarea>
  <select name="pais" ngModel>
    <option value="es">España</option>
    <option value="it">Italia</option>
  </select>
  <label><input type="checkbox" name="suscrito" ngModel> Suscribirse</label>
  <button type="submit">Enviar</button>
</form>
```

1.4 Validación

En los formularios de plantilla, Angular aplica validaciones directamente desde el **HTML**, usando atributos estándar y directivas como ngModel.

Además, Angular genera estados automáticos: valid, invalid, touched, dirty, etc.

```
<form #f="ngForm" (ngSubmit)="enviar(f)">

  <label>Nombre:</label>
  <input name="nombre" ngModel required minlength="3"
#n="ngModel">
  <div *ngIf="n.invalid && n.touched">
    Debe tener al menos 3 caracteres.
  </div>

  <label>Edad:</label>
  <input type="number" name="edad" ngModel min="18" max="65"
#e="ngModel">
  <div *ngIf="e.invalid && e.touched">
    La edad debe estar entre 18 y 65.
  </div>

  <label>Email:</label>
  <input type="email" name="correo" ngModel email #c="ngModel">
  <div *ngIf="c.invalid && c.touched">
    Email no válido.
  </div>

  <button type="submit" [disabled]="f.invalid">Enviar</button>
</form>
```

La validación personalizada con pattern

```
<input
  name="dni"
  ngModel
  pattern="^[0-9]{8}[A-Za-z]$"
  #dni="ngModel">

<div *ngIf="dni.invalid && dni.touched">
  DNI no válido.
</div>
```

1.5 Creando nuevos validadores

Además de las validaciones estándar (*required*, *email*, *minlength,* …), Angular permite crear validadores personalizados para formularios de plantilla (*template-driven*) y reactivos.

En formularios de plantilla, los validadores personalizados se crean como **directivas**.

Crear un validador personalizado (directiva)

Paso 1: Crear la directiva

```
import { Directive } from '@angular/core';
import { NG_VALIDATORS, Validator, AbstractControl, ValidationErrors
} from '@angular/forms';

@Directive({
    selector: '[sinNumeros]',
    providers: [
      { provide: NG_VALIDATORS,
        useExisting: SinNumerosValidatorDirective,
        multi: true
      }
   ]
})

export class SinNumerosValidatorDirective implements Validator {
  validate(control: AbstractControl): ValidationErrors | null {
     const tieneNumero = /\d/.test(control.value);
     return tieneNumero ? { sinNumeros: true } : null;
  }
}
```

Paso 2. Usar el validador en un componente standalone

En el componente:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { SinNumerosValidatorDirective } from './sin-
numeros.directive';

@Component({
    selector: 'app-formulario',
    standalone: true,
    imports: [FormsModule, SinNumerosValidatorDirective],
    templateUrl: './formulario.component.html'
})
```

```
export class FormularioComponent {
   nombre = '';
}
```

Paso 3. Aplicar el validador en la plantilla

formulario.component.html

```
<input
  name="nombre"
  ngModel
  sinNumeros
  #n="ngModel">

<div *ngIf="n.errors?.['sinNumeros'] && n.touched">
  El nombre no puede contener números.
</div>
```

Paso 4. Ejemplo de otro validador: mínimo de palabras

Directiva

```
@Directive({
     selector: '[minPalabras]',
     providers: [
        { provide: NG_VALIDATORS,
          useExisting: MinPalabrasDirective,
          multi: true
        }
     ]
})

export class MinPalabrasDirective implements Validator {
   validate(control: AbstractControl): ValidationErrors | null {
      const palabras = control.value?.trim().split(/\s+/) || [];
      return palabras.length < 3 ? { minPalabras: true } : null;
   }
}
```

Uso en HTML

```
<textarea name="descripcion" ngModel minPalabras #d="ngModel">
</textarea>

<div *ngIf="d.errors?.['minPalabras'] && d.touched">
  Debe escribir al menos 3 palabras.
</div>
```

1.6 Enviando el formulario

Paso 1. Preparar el componente standalone

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
    selector: 'app-formulario',
    standalone: true,
    imports: [FormsModule],
    templateUrl: './formulario.component.html'
})
export class FormularioComponent {
  enviar(form: any) {
    console.log('Datos enviados:', form.value);
  }
}
```

Paso 2. Enviar el formulario desde la plantilla

```
formulario.component.html
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <label>Nombre:</label>
  <input name="nombre" ngModel required>

  <label>Email:</label>
  <input name="email" ngModel email>

  <button type="submit">Enviar</button>
</form>
```

Paso 3. Validar antes de enviar

Puedes desactivar el botón si el formulario es inválido:

```
<button type="submit" [disabled]="f.invalid">Enviar</button>
```

Paso 4. Acceder a los datos enviados

En el componente:

```
enviar(form: any) {
 console.log(form.value); // { nombre: '...', email: '...' }
 console.log(form.valid); // true / false
}
```

1.7 Validar antes de enviar

En los formularios de plantilla, Angular permite **bloquear el envío** si el formulario o alguno de sus campos es inválido.

La validación se hace directamente desde el HTML usando *ngForm* y *ngModel*.

Desactivar el botón si el formulario es inválido

```
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <input name="nombre" ngModel required>
  <input name="email" ngModel email>

  <button type="submit" [disabled]="f.invalid">Enviar</button>
</form>
```

Mostrar errores antes de permitir el envío

```
<input
  name="nombre"
  ngModel
  required
  minlength="3"
  #n="ngModel">

<div *ngIf="n.invalid && n.touched">
  El nombre es obligatorio y debe tener al menos 3 caracteres.
</div>
```

Validar dentro del método enviar()

```
enviar(form: any) {
  if (form.invalid) {
    console.log('Formulario inválido, no se envía');
    return;
  }

  console.log('Datos enviados:', form.value);
}
```

Un ejemplo completo y funcional

HTML

```
<form #f="ngForm" (ngSubmit)="enviar(f)">
  <label>Nombre:</label>
  <input name="nombre" ngModel required minlength="3" #n="ngModel">
  <div *ngIf="n.invalid && n.touched">Nombre mínimo 3 caracteres.</div>

  <label>Email:</label>
  <input name="email" ngModel email #e="ngModel">
  <div *ngIf="e.invalid && e.touched">Email no válido.</div>

  <button type="submit" [disabled]="f.invalid">Enviar</button>
</form>
```

TS

```
enviar(form: any) {
  if (form.invalid) return;
  console.log('Formulario enviado:', form.value);
}
```

1.8 Subir un archivo en el objeto JSON

Plantilla HTML con campo de archivo

```
<form #f="ngForm" (ngSubmit)="enviar()">
  <input name="nombre" [(ngModel)]="datos.nombre">

  <input type="file" (change)="seleccionarArchivo($event)">

  <button type="submit">Enviar</button>
</form>
```

Componente standalone

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-subida',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './subida.component.html'
})
export class SubidaComponent {
```

```
  datos = { nombre: '' };
  archivo!: File;

  constructor(private http: HttpClient) {}

  seleccionarArchivo(event: any) {
    this.archivo = event.target.files[0];
  }

  enviar() {
    const formData = new FormData();

    // 1. Añadir el JSON como string
    formData.append('datos', JSON.stringify(this.datos));

    // 2. Añadir el archivo
    formData.append('archivo', this.archivo);

    // 3. Enviar al backend
    this.http.post('/api/subida', formData).subscribe(r => {
      console.log('Enviado correctamente');
    });
  }
}
```

2 Formularios reactivos

2.1 Creando un formulario Reactivo

Los **formularios reactivos** en Angular 20 permiten manejar datos de forma estructurada, validada y totalmente programática. Son ideales cuando necesitas control total sobre el estado del formulario.

src/app/form-reactivo/form-reactivo.component.ts

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators }
from '@angular/forms';

@Component({
  selector: 'app-form-reactivo',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './form-reactivo.component.html',
  styleUrls: ['./form-reactivo.component.css']
})
export class FormReactivoComponent {
```

```
  formulario = new FormGroup({
    nombre: new FormControl('', [
      Validators.required,
      Validators.minLength(3)
    ]),
    email: new FormControl('', [
      Validators.required,
      Validators.email
    ]),
    edad: new FormControl(null, [
      Validators.required,
      Validators.min(18)
    ])
  });

  enviar() {
    if (this.formulario.valid) {
      console.log('Datos enviados:', this.formulario.value);
      alert('Formulario enviado correctamente');
      this.formulario.reset();
    }
  }
}
```

src/app/form-reactivo/form-reactivo.component.html

```
<h2>Formulario Reactivo</h2>

<form [formGroup]="formulario" (ngSubmit)="enviar()">

  <!-- Nombre -->
  <label>Nombre</label>
  <input type="text" formControlName="nombre">
  <div class="error" *ngIf="formulario.get('nombre')?.touched &&
formulario.get('nombre')?.invalid">
    <span *ngIf="formulario.get('nombre')?.errors?.['required']">El
nombre es obligatorio.</span>
    <span
*ngIf="formulario.get('nombre')?.errors?.['minlength']">Debe tener
al menos 3 caracteres.</span>
  </div>

  <!-- Email -->
  <label>Email</label>
  <input type="email" formControlName="email">
  <div class="error" *ngIf="formulario.get('email')?.touched &&
formulario.get('email')?.invalid">
    <span *ngIf="formulario.get('email')?.errors?.['required']">El
email es obligatorio.</span>
```

```
    <span *ngIf="formulario.get('email')?.errors?.['email']">Formato
de email no válido.</span>
  </div>

  <!-- Edad -->
  <label>Edad</label>
  <input type="number" formControlName="edad">
  <div class="error" *ngIf="formulario.get('edad')?.touched &&
formulario.get('edad')?.invalid">
    <span *ngIf="formulario.get('edad')?.errors?.['required']">La
edad es obligatoria.</span>
    <span *ngIf="formulario.get('edad')?.errors?.['min']">Debe ser
mayor o igual a 18.</span>
  </div>

  <button type="submit"
[disabled]="formulario.invalid">Enviar</button>
</form>
```

src/app/form-reactivo/form-reactivo.component.css

```
form {
  display: flex;
  flex-direction: column;
  width: 300px;
  gap: 12px;
}

input {
  padding: 6px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

button {
  padding: 8px;
  background: #1976d2;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

button:disabled {
  background: #9e9e9e;
  cursor: not-allowed;
}

.error {
```

```
  color: #d32f2f;
  font-size: 0.85rem;
}
```

2.2 Crear nuevos validadores

Además de las validaciones integradas (*required,email,minLength*, etc.), Angular permite crear validadores personalizados para reglas específicas, como validar un DNI, un rango de edad o que un nombre no contenga números.

Un validador personalizado es simplemente una función que devuelve:

- *null* → si el valor es válido
- un objeto con el error → si es inválido

Sin-numeros.validator.ts

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

export function sinNumerosValidator(control: AbstractControl):
ValidationErrors | null {
  const tieneNumero = /\d/.test(control.value);
  return tieneNumero ? { contieneNumeros: true } : null;
}
```

Validadores.component.ts

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators }
from '@angular/forms';
import { sinNumerosValidator } from './sin-numeros.validator';

@Component({
  selector: 'app-validadores',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './validadores.component.html'
})
export class ValidadoresComponent {

  formulario = new FormGroup({
    nombre: new FormControl('', [
      Validators.required,
      sinNumerosValidator
    ])
  });

  enviar() {
```

```
    if (this.formulario.valid) {
      console.log('Datos válidos:', this.formulario.value);
      alert('Formulario enviado');
    }
  }
}
```

Validadores.component.html

```
<h2>Validador Personalizado</h2>

<form [formGroup]="formulario" (ngSubmit)="enviar()">

  <label>Nombre</label>
  <input type="text" formControlName="nombre">

  <div class="error" *ngIf="formulario.get('nombre')?.touched &&
formulario.get('nombre')?.invalid">
    <span *ngIf="formulario.get('nombre')?.errors?.['required']">El
nombre es obligatorio.</span>
    <span
*ngIf="formulario.get('nombre')?.errors?.['contieneNumeros']">No
puede contener números.</span>
  </div>

  <button type="submit"
[disabled]="formulario.invalid">Enviar</button>
</form>
```

2.3 Agrupando campos

En Angular 20, puedes agrupar campos relacionados dentro de un formulario usando *FormGroup*. Esto permite organizar mejor la información, validar grupos completos y manejar estructuras más complejas (por ejemplo, dirección, datos personales, credenciales).

Agrupando.component.ts

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators }
from '@angular/forms';

@Component({
  selector: 'app-agrupando',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './agrupando.component.html'
})
export class AgrupandoComponent {
```

```
  formulario = new FormGroup({
    nombre: new FormControl('', Validators.required),

    direccion: new FormGroup({
      calle: new FormControl('', Validators.required),
      ciudad: new FormControl('', Validators.required),
      cp: new FormControl('', [
        Validators.required,
        Validators.pattern(/^[0-9]{5}$/)
      ])
    })
  });

  enviar() {
    if (this.formulario.valid) {
      console.log('Datos enviados:', this.formulario.value);
      alert('Formulario enviado correctamente');
    }
  }
}
```

Agrupando.component.html

```
<h2>Agrupando Campos</h2>

<form [formGroup]="formulario" (ngSubmit)="enviar()">

  <!-- Nombre -->
  <label>Nombre</label>
  <input type="text" formControlName="nombre">
  <div class="error" *ngIf="formulario.get('nombre')?.touched &&
formulario.get('nombre')?.invalid">
    El nombre es obligatorio.
  </div>

  <!-- Grupo Dirección -->
  <fieldset formGroupName="direccion">
    <legend>Dirección</legend>

    <label>Calle</label>
    <input type="text" formControlName="calle">

    <label>Ciudad</label>
    <input type="text" formControlName="ciudad">

    <label>Código Postal</label>
    <input type="text" formControlName="cp">
```

```
    <div class="error" *ngIf="formulario.get('direccion.cp')?.touched
&& formulario.get('direccion.cp')?.invalid">
      Código postal inválido (5 números).
    </div>
  </fieldset>

  <button type="submit"
[disabled]="formulario.invalid">Enviar</button>
</form>
```

Agrupando.component.css

```
form {
  display: flex;
  flex-direction: column;
  width: 320px;
  gap: 12px;
}

fieldset {
  border: 1px solid #ccc;
  padding: 10px;
  border-radius: 6px;
}

.error {
  color: #d32f2f;
  font-size: 0.85rem;
}
```

2.4 Reaccionando ante cambios

Los formularios reactivos permiten **escuchar cambios en tiempo real** usando:

- *valueChanges* → detecta cambios en el valor
- *statusChanges* → detecta cambios en el estado (VALID, INVALID, etc.)

Esto es útil para validar dinámicamente, activar/desactivar campos, mostrar mensajes o enviar datos en vivo.

Reaccion-cambios.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, Validators }
from '@angular/forms';

@Component({
```

```
  selector: 'app-reaccion-cambios',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './reaccion-cambios.component.html'
})
export class ReaccionCambiosComponent implements OnInit {

  formulario = new FormGroup({
    nombre: new FormControl('', Validators.required),
    edad: new FormControl(18, Validators.min(18))
  });

  ngOnInit() {
    // Reaccionar a cambios en el valor
    this.formulario.get('nombre')?.valueChanges.subscribe(valor => {
      console.log('Nombre cambió a:', valor);
    });

    // Reaccionar a cambios en el estado del formulario
    this.formulario.statusChanges.subscribe(estado => {
      console.log('Estado del formulario:', estado);
    });
  }

  enviar() {
    console.log('Datos enviados:', this.formulario.value);
  }
}
```

Reaccion-cambios.component.html

```
<h2>Reaccionando ante Cambios</h2>

<form [formGroup]="formulario" (ngSubmit)="enviar()">

  <label>Nombre</label>
  <input type="text" formControlName="nombre">

  <label>Edad</label>
  <input type="number" formControlName="edad">

  <button type="submit">Enviar</button>
</form>
```

2.5 Duplicando campos (FormArray)

FormArray permite **crear grupos de campos dinámicos**, como listas de teléfonos, emails, direcciones o productos. Puedes **añadir, eliminar o recorrer** elementos fácilmente.

Form-array.component.ts

```
import { Component } from '@angular/core';
import { ReactiveFormsModule, FormGroup, FormControl, FormArray,
Validators } from '@angular/forms';

@Component({
  selector: 'app-form-array',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './form-array.component.html'
})
export class FormArrayComponent {

  formulario = new FormGroup({
    nombre: new FormControl('', Validators.required),
    telefonos: new FormArray([
      new FormControl('', Validators.required)
    ])
  });

  get telefonos() {
    return this.formulario.get('telefonos') as FormArray;
  }

  agregarTelefono() {
    this.telefonos.push(new FormControl('', Validators.required));
  }

  eliminarTelefono(i: number) {
    this.telefonos.removeAt(i);
  }

  enviar() {
    console.log('Datos enviados:', this.formulario.value);
  }
}
```

Form-array.component.html

```
<h2>Duplicando Campos con FormArray</h2>

<form [formGroup]="formulario" (ngSubmit)="enviar()">

  <label>Nombre</label>
  <input type="text" formControlName="nombre">

  <h3>Teléfonos</h3>

  <div formArrayName="telefonos">
    <div *ngFor="let tel of telefonos.controls; let i = index">
      <input [formControlName]="i" placeholder="Teléfono">

      <button type="button" (click)="eliminarTelefono(i)"
*ngIf="telefonos.length > 1">
        Eliminar
      </button>
    </div>
  </div>

  <button type="button" (click)="agregarTelefono()">Añadir
teléfono</button>

  <button type="submit"
[disabled]="formulario.invalid">Enviar</button>
</form>
```